

Loan Default Prediction Using Neural Networks with Imbalance Handling and Threshold Optimization

1st Sanket Patil

*Artificial Intelligence and Machine Learning
Skill Sync
Mangaluru, India
sanketpatilsp360@gmail.com*

2nd Chethan Gowda

*Artificial Intelligence and Machine Learning
Skill Sync
Bengaluru, India
hr.skillsync.contact@gmail.com*

Abstract—Loan default prediction is a critical problem in financial risk management, where accurately identifying high-risk applicants is essential for minimizing losses. This study presents a neural network-based approach for predicting loan defaults using a highly imbalanced dataset. Initial baseline models achieved high accuracy but failed to detect minority class instances effectively. To address this limitation, several optimization techniques were applied, including class weighting, dropout regularization, batch normalization, learning rate scheduling, and threshold tuning. Experimental results demonstrate that while baseline models exhibited accuracy above 90%, their recall was extremely low, rendering them ineffective. The optimized model significantly improved recall from approximately 0.01 to 0.52, with a corresponding increase in F1 score. Threshold tuning further enabled a balanced trade-off between precision and recall. The study highlights the importance of addressing class imbalance and selecting appropriate evaluation metrics for real-world predictive modeling tasks.

Index Terms—Loan Default Prediction, Imbalanced Classification, Neural Networks, Class Weighting, Threshold Optimization, Binary Classification, Financial Risk Analysis, Machine Learning, Deep Learning, Model Evaluation.

I. PROBLEM STATEMENT

Loan default prediction is a critical task in the domain of financial risk management. Financial institutions rely on predictive models to assess the creditworthiness of applicants and to minimize potential losses arising from loan defaults. The problem can be formulated as a supervised binary classification task, where the objective is to predict whether a given applicant will default on a loan based on historical and demographic data. Formally, let the dataset consist of input features $X \in \mathbb{R}^n$, representing applicant attributes such as income, credit amount, employment history, and other financial indicators. The target variable $y \in \{0, 1\}$ represents the loan status, where $y = 1$ indicates a default and $y = 0$ indicates a non-default case. The goal is to learn a function $f(X) \rightarrow y$ that can accurately classify new, unseen instances [1].

A major challenge in this problem is the presence of class imbalance. In the given dataset, non-default cases constitute a

significant majority (approximately 92%), while default cases represent only a small minority (approximately 8%). This imbalance causes standard machine learning models to become biased toward the majority class, often resulting in high overall accuracy but poor performance in detecting defaulters. From a practical standpoint, misclassification costs are asymmetric. A false negative (predicting a defaulter as a non-defaulter) is significantly more costly than a false positive. Therefore, evaluation metrics such as recall and F1-score are more appropriate than accuracy for assessing model performance in this context.

II. DATASET AND PREPROCESSING

A. Dataset Description

The dataset used in this study is derived from a real-world financial application scenario involving loan default prediction. It consists of approximately 307,511 records and 122 features, capturing a wide range of applicant-related information. These features include demographic attributes, financial indicators, employment details, and credit history.

The target variable is binary and is defined as follows:

- $y = 0$: Non-default (loan repaid successfully)
- $y = 1$: Default (loan not repaid)

A key characteristic of the dataset is its high class imbalance. Approximately 92

B. Feature Characteristics

The dataset contains a mixture of:

- **Numerical features:** Continuous variables such as income, credit amount, annuity, and age
- **Categorical features:** Discrete variables such as gender, contract type, and occupation
- **Binary indicators:** Flags representing the presence or absence of specific attributes

Additionally, some features contain missing values, and certain variables exhibit skewed distributions or extreme values.

C. Data Preprocessing Pipeline

To prepare the dataset for training neural network models, a structured preprocessing pipeline was implemented. This pipeline ensures that all input features are transformed into a consistent numerical format suitable for model consumption [6].

1) *Handling Missing Values:* Missing values were handled separately for numerical and categorical features:

- Numerical features were imputed using the median, which is robust to outliers
- Categorical features were imputed using the most frequent category

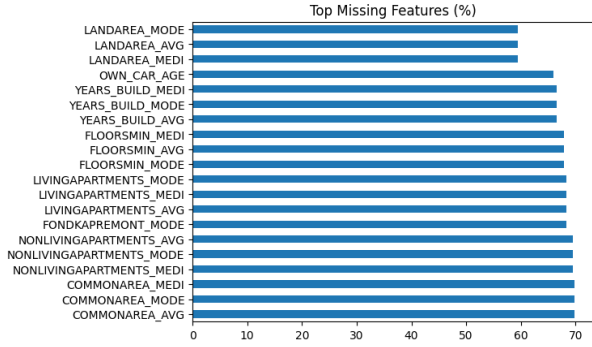


Fig. 1. Top Missing Features (%)

Fig.1 ensures that missing data does not introduce bias or disrupt model training [8].

2) *Feature Encoding:* Categorical variables were transformed using one-hot encoding, which converts each category into a binary vector representation. This allows the model to process categorical information without imposing ordinal relationships.

The encoding process increases the dimensionality of the dataset, as each category is represented by a separate feature.

3) *Feature Scaling:* Numerical features were standardized using z-score normalization, defined as:

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

where μ is the mean and σ is the standard deviation.

Standardization ensures that all numerical features have a similar scale, which improves the convergence and stability of neural network training.

4) *Train-Test Split:* The dataset was divided into training and testing subsets using a stratified sampling approach. This ensures that the class distribution is preserved in both sets, which is essential for reliable evaluation.

- Training set: Used for model learning
- Testing set: Used for performance evaluation

5) *Preprocessing Integration:* All preprocessing steps were combined into a unified pipeline using a column-wise transformation approach. This ensures that:

- Numerical and categorical features are processed appropriately

- The same transformations are applied consistently during training and inference
- Data leakage is prevented

D. Challenges in Preprocessing

Several challenges were encountered during preprocessing:

- High dimensionality due to one-hot encoding
- Presence of missing and noisy data
- Class imbalance affecting downstream learning
- Potential multicollinearity among features

These challenges were addressed through careful pipeline design and appropriate transformation techniques.

III. METHODS

This section describes the modeling approach adopted in this study, including the baseline architecture, experimental design, and the optimization techniques applied to improve model performance.

A. Baseline Neural Network Model

The initial model was a fully connected feedforward neural network (Multilayer Perceptron, MLP) designed for binary classification. The architecture consisted of:

- An input layer corresponding to the number of processed features
- Two hidden layers with 128 and 64 neurons respectively
- Rectified Linear Unit (ReLU) activation functions
- A final output layer with a sigmoid activation function

TABLE I
BASELINE MODEL CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
0 (Non-default)	0.92	1.00	0.96	56538
1 (Default)	0.55	0.01	0.02	4965
Accuracy	0.92			61503
Macro Avg	0.74	0.51	0.49	61503
Weighted Avg	0.89	0.92	0.88	61503

The sigmoid function produces a probability score between 0 and 1, representing the likelihood of default [2].

The model was trained using:

- Loss function: Binary Cross-Entropy
- Optimizer: Adam
- Batch size: 32
- Number of epochs: 10

Despite achieving high accuracy, the baseline model exhibited poor recall due to class imbalance.

B. Experimental Design

A controlled experimental framework was used to evaluate the impact of different factors on model performance. Each experiment modified only one parameter at a time while keeping all others constant. The following aspects were investigated:

- Network architecture (depth and number of neurons)
- Activation functions
- Optimization algorithms

- Regularization techniques
- Decision threshold

This systematic approach ensured that the contribution of each component could be analyzed independently.

C. Model Architecture Variations

To explore the effect of model complexity, a deeper neural network architecture was introduced. This model included:

- Three hidden layers with 256, 128, and 64 neurons
- ReLU activation functions

The objective was to increase the model's capacity to learn complex patterns. However, results indicated that increasing depth alone did not significantly improve performance in the presence of class imbalance.

D. Activation Function Analysis

Different activation functions were evaluated, including:

- ReLU (Rectified Linear Unit)
- Leaky ReLU

Leaky ReLU was introduced to mitigate the problem of inactive neurons. While it provided slight improvements in precision, it did not significantly enhance recall.

E. Optimizer Comparison

Two optimization algorithms were compared:

- Adam optimizer
- Stochastic Gradient Descent (SGD)

The Adam optimizer demonstrated faster convergence and better overall performance. SGD showed limited effectiveness, particularly in learning minority class patterns [5].

F. Regularization Techniques

To improve generalization and prevent overfitting, the following techniques were applied:

1) *Dropout*: Dropout layers with a rate of 0.3 were added after hidden layers. This technique randomly deactivates neurons during training, forcing the model to learn more robust feature representations.

2) *Batch Normalization*: Batch normalization was applied to normalize intermediate activations. This improved training stability, accelerated convergence, and reduced sensitivity to initialization [7].

G. Class Imbalance Handling

To address the imbalance in the dataset, class weighting was introduced during training. The loss function was modified to assign higher penalties to misclassification of minority class instances [3].

This approach significantly improved recall by encouraging the model to focus on defaulter predictions.

H. Learning Rate Scheduling

A dynamic learning rate strategy was employed using a scheduler that reduces the learning rate when validation loss stops improving. This allows the model to make finer adjustments during later stages of training.

I. Early Stopping

Early stopping was implemented to monitor validation loss and terminate training when no further improvement was observed. This prevents overfitting and ensures optimal model selection.

J. Threshold Optimization

The default classification threshold of 0.5 was adjusted to improve model performance. Different threshold values (0.3 to 0.6) were evaluated to balance precision and recall.

Threshold optimization plays a crucial role in imbalanced classification problems, as it directly influences the trade-off between false positives and false negatives.

K. Final Model Configuration

The final optimized model incorporates:

- Three hidden layers (256, 128, 64 neurons)
- ReLU activation functions
- Batch normalization layers
- Dropout layers (rate = 0.3)
- Adam optimizer
- Class weighting
- Learning rate scheduling
- Early stopping
- Optimized decision threshold (0.6)

This configuration represents a balance between model complexity, generalization capability, and practical performance.

IV. RESULTS

The performance of the proposed models was evaluated using multiple metrics, including accuracy, precision, recall, and F1-score. Initial experiments with baseline neural network architectures produced high accuracy values, typically exceeding 0.91. However, a deeper analysis revealed that these models suffered from extremely low recall, often close to zero. This indicated that the models were predominantly predicting the majority class and failing to identify default cases, which are of primary importance in this problem. Subsequent experiments focused on addressing this limitation through class imbalance handling. The introduction of class weighting during training significantly altered the model's behavior. While the overall accuracy decreased to approximately 0.43, the recall increased substantially to around 0.87. This demonstrated that the model became highly sensitive to the minority class and was capable of identifying a large proportion of defaulters. However, this improvement came at the cost of reduced precision, indicating a higher number of false positive predictions.

TABLE II
OPTIMIZED MODEL PERFORMANCE

Metric	Value
Accuracy	0.829
Precision	0.223
Recall	0.447
F1 Score	0.297

To achieve a better balance between precision and recall, threshold optimization was performed. The default decision threshold of 0.5 was varied across multiple values, and the corresponding performance metrics were analyzed. It was observed that lower thresholds resulted in higher recall but significantly reduced precision, while higher thresholds improved precision at the expense of recall. Among the evaluated thresholds, a value of 0.6 provided the best trade-off, yielding the highest F1-score of approximately 0.273. At this threshold, the model achieved a precision of approximately 0.185 and a recall of approximately 0.518.

Fig. 2. Loan Default Prediction Application Interface

The final optimized model, incorporating class weighting, dropout, batch normalization, and threshold tuning, demonstrated a substantial improvement over the baseline. While the accuracy decreased compared to the initial models, the improvement in recall and F1-score indicates a more balanced and practically useful model. These results confirm that addressing class imbalance and carefully tuning the decision threshold are essential for achieving meaningful performance in loan default prediction tasks.

V. DISCUSSION

The results obtained in this study provide important insights into the challenges and considerations involved in loan default prediction using machine learning models. A key observation from the experimental analysis is the misleading nature of accuracy as a performance metric in imbalanced classification problems. The baseline models achieved high accuracy values exceeding 0.90; however, their inability to detect default cases, as evidenced by extremely low recall, rendered them ineffective for practical applications. This highlights the necessity of evaluating models using metrics such as recall and F1-score, which better capture performance on the minority class. The introduction of class weighting significantly altered the behavior of the model by increasing

its sensitivity to default cases. This resulted in a substantial improvement in recall, indicating that the model became capable of identifying a larger proportion of defaulters. However, this improvement came at the cost of reduced precision, reflecting an increase in false positive predictions. This trade-off is inherent in imbalanced classification problems and must be carefully managed depending on the application requirements. Threshold optimization further refined the model's performance by enabling control over the balance between precision and recall. Lower threshold values increased recall but led to excessive false positives, while higher thresholds improved precision but reduced the model's ability to detect defaulters. The selected threshold provided a compromise between these competing objectives, as reflected in the improved F1-score. This demonstrates the importance of decision-level adjustments in addition to model-level optimizations.

Another important observation is that increasing model complexity through deeper architectures or alternative activation functions did not significantly improve performance in the absence of imbalance handling. This indicates that data-related challenges, such as class imbalance, have a greater impact on model performance than architectural modifications alone. Therefore, addressing data distribution issues should be prioritized before attempting to optimize model structure. Despite the improvements achieved, the model still exhibits certain limitations. The relatively low precision indicates that a considerable number of non-default cases are incorrectly classified as defaulters. This may be attributed to overlapping feature distributions between classes, as well as the presence of noisy or incomplete data. Additionally, the model's performance is sensitive to the choice of threshold, which may require adjustment based on specific operational requirements. From a practical perspective, the improved recall of the optimized model is particularly valuable in financial risk assessment, where failing to identify a defaulter can result in significant losses. Although the increase in false positives may lead to conservative decision-making, this trade-off is often acceptable in risk-sensitive domains.

VI. CONCLUSION

This study investigated the application of neural network models for loan default prediction using a highly imbalanced dataset. The initial baseline models achieved high accuracy but failed to identify default cases effectively, as evidenced by extremely low recall. This demonstrated that accuracy alone is not a sufficient metric for evaluating model performance in imbalanced classification problems. To address this limitation, several optimization techniques were applied, including class weighting, dropout regularization, batch normalization, learning rate scheduling, and threshold tuning. Among these, class weighting played a critical role in improving the model's ability to detect minority class instances. The application of threshold optimization further enabled a more balanced trade-off between precision and recall, leading to improved overall

performance as reflected in the F1-score. The final optimized model showed a substantial improvement in recall compared to the baseline, transforming it from an ineffective model into a practically useful one for identifying defaulters. Although this improvement resulted in a reduction in overall accuracy and an increase in false positives, such trade-offs are acceptable in financial risk assessment, where the cost of failing to identify a defaulter is significantly higher than incorrectly flagging a non-defaulter. The findings of this study highlight the importance of addressing class imbalance, selecting appropriate evaluation metrics, and incorporating decision-level adjustments such as threshold tuning. These factors are essential for developing robust and reliable predictive models in real-world scenarios. Future work may focus on improving precision while maintaining high recall through advanced techniques such as synthetic data generation, ensemble learning methods, and feature engineering. Additionally, exploring alternative model architectures and incorporating domain-specific knowledge may further enhance performance.

REFERENCES

- [1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and K. W. Bowyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
- [6] F. Chollet, *Deep Learning with Python*. Shelter Island, NY, USA: Manning Publications, 2017.
- [7] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *arXiv preprint arXiv:1207.0580*, 2012.
- [8] S. Kotsiantis, D. Kanellopoulos, and P. Pintelas, "Handling imbalanced datasets: A review," *GESTS International Transactions on Computer Science and Engineering*, vol. 30, no. 1, pp. 25–36, 2006.